Actor和CSP设计模式

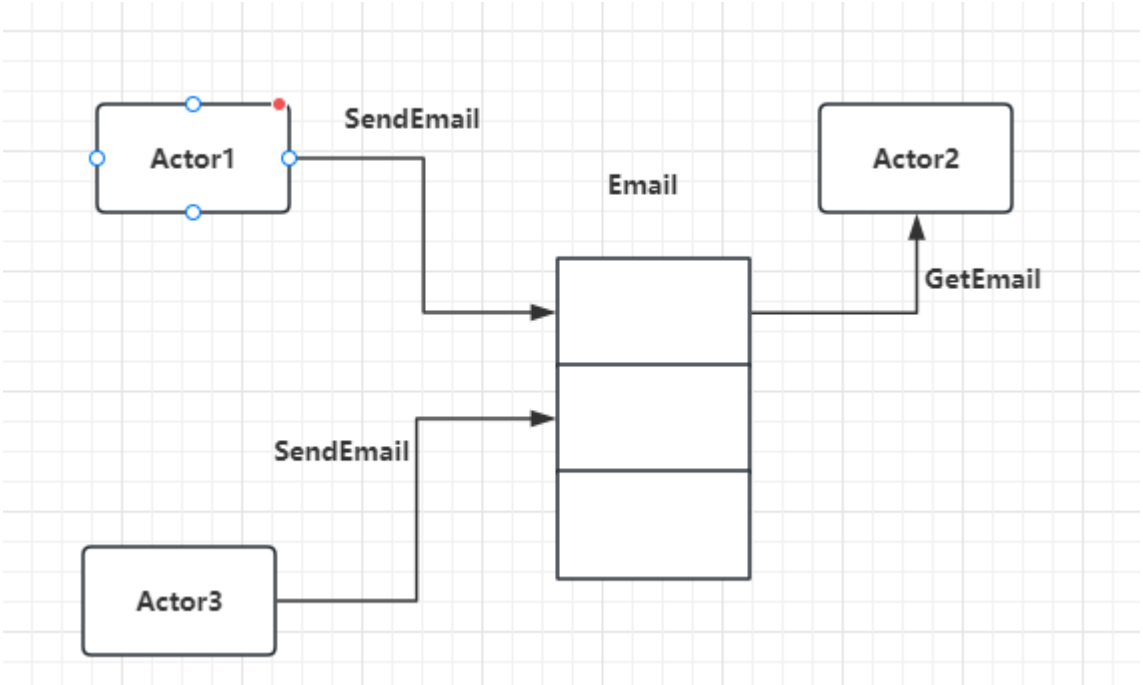
Posted on 2023-10-06 11:36:55 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2WNBz17Q9PNiHXiLygjre8TdhlL)

简介

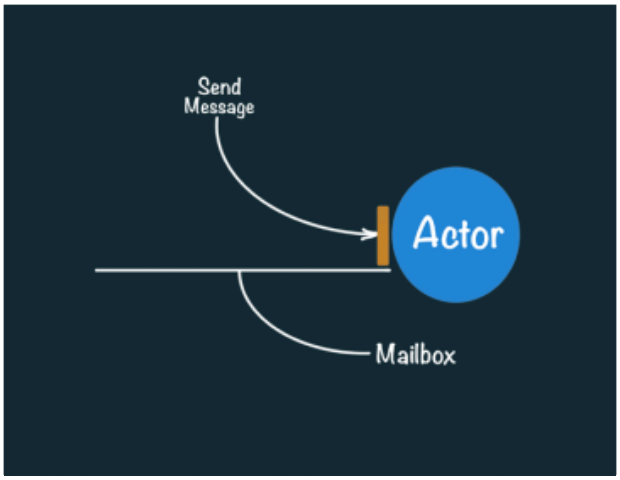
本文介绍两种并发设计中常用的设计模式，包括Actor和CSP模式。传统的并发设计经常都是通过共享内存加锁保证逻辑安全，这种模式有几个缺点，包括1 频繁加锁影响性能，2 耦合度高。后来大家提出了Actor和CSP设计模式。

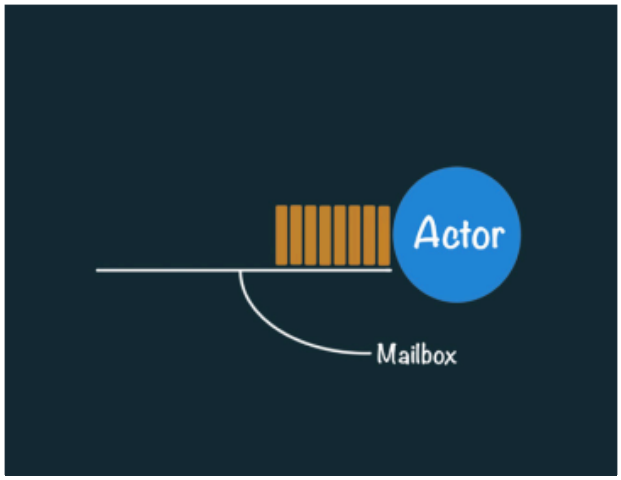
Actor设计模式

简单点说，actor通过消息传递的方式与外界通信。消息传递是异步的。每个actor都有一个邮箱，该邮箱接收并缓存其他actor发过来的消息，actor一次只能同步处理一个消息，处理消息过程中，除了可以接收消息，不能做任何其他操作。
每一个类独立在一个线程里称作Actor，Actor之间通过队列通信，比如Actor1 发消息给Actor2， Actor2 发消息给Actor1都是投递到对方的队列中。好像给对方发邮件，对方从邮箱中取出一样。如下图



Actor模型的另一个好处就是可以消除共享状态，因为它每次只能处理一条消息，所以actor内部可以安全的处理状态，而不用考虑锁机制。





我们在网络编程中对于逻辑层的处理就采用了将要处理的逻辑消息封装成包投递给逻辑队列，逻辑类从队列中消费的思想，其实就是一种Actor设计模式。Erlang是天然支持Actor的语言。

CSP模式

CSP 是 Communicating Sequential Process 的简称，中文可以叫做通信顺序进程，是一种并发编程模型，是一个很强大的并发数据模型，是上个世纪七十年代提出的，用于描述两个独立的并发实体通过共享的通讯 channel(管道)进行通信的并发模型。相对于Actor模型，CSP中channel是第一类对象，它不关注发送消息的实体，而关注与发送消息时使用的channel。go是天然支持csp模式的语言。

CSP和Actor类似，只不过CSP将消息投递给channel，至于谁从channel中取数据，发送的一方是不关注的。简单的说Actor在发送消息前是直到接收方是谁，而接受方收到消息后也知道发送方是谁，更像是邮件的通信模式。而csp是完全解耦合的。



无论Actor还是CSP，他们都有一个共同的特性”Do not communicate by sharing memory; instead, share memory by communicating”

go风格的csp

我们通过生产者和消费者模型给大家演示csp模式的使用方式，用go来做示例

```
1. package main
2.
3. import (
4.     "cspdemo/message"
5.     "fmt"
6.     "os"
7.     "os/signal"
8.     "syscall"
9. )
10.
11. var closeChan chan struct{}
12. var sigs chan os.Signal
13.
14. func init() {
15.     //类似于auto
16.     sigs = make(chan os.Signal)
17.     //具体类型初始化
18.     closeChan = make(chan struct{})
19.     signal.Notify(sigs, syscall.SIGINT, syscall.SIGTERM)
20.     //可以理解为C++ 的匿名函数，或者js的匿名函数，此处通过go原语启动一个协程并行执行
21.     go func() {
22.         sig := <-sigs
23.         fmt.Println("receive signal is ", sig)
24.         close(closeChan)
25.         message.ConsumerInst().Exit()
26.         message.ProducerInst().Exit()
27.     }()
28. }
29.
30. func main() {
31.
32.     fmt.Println("Main Process begin!")
33.     <-closeChan
34.     message.ConsumerInst().Join()
35.     message.ProducerInst().Join()
36.     fmt.Println("Main Process exit!")
37. }
```

在上面的代码中我们启动了一个协程监听Ctrl+C等退出操作，当收到Ctrl+C的信号后，会关闭closeChan这个channel。这样主函数中 `<-closeChan` 就会从channel中取出数据。然后等待消费者和生产者退出。

接下来我们将生产者和消费者的实现放入message包，先看下message公共数据的定义

```
1. package message
2.
3. const MAX_COUNT = 200
4.
5. var msgChan = make(chan int, MAX_COUNT)
```

上面的代码中我们定义了一个channel，大小为200，大家可以理解为仓库的大小为200，生产者向仓库中投递数据如果达到200就会阻塞。直到有消费者从中消费数据，如果消费者发现channel中数据为0，则阻塞。

```
1. package message
2.
3. import (
4.     "context"
5.     "fmt"
6.     "sync"
7.     "time"
8. )
9.
10. var producer *Producer = nil
11. var producer_once sync.Once
12.
13. func init() {
14.     // Consumer1 = new(Consumer)
15.     //类似于C++ std::call_once
16.     producer_once.Do(func() {
17.         producer = new(Producer)
18.         producer._exited = make(chan struct{})
19.         producer._ctx, producer._cancel = context.WithCancel(context.Background())
20.         producer.StartWork()
21.     })
22. }
23.
24. func ProducerInst() *Producer {
25.     return producer
26. }
27.
28. type Producer struct {
29.     _exited chan struct{}
30.     _ctx    context.Context
31.     _cancel context.CancelFunc
32. }
33.
34. func (producer *Producer) Join() {
35.     <-producer._exited
36.     fmt.Println("producer exited")
37. }
38.
39. func (producer *Producer) Exit() {
40.     producer._cancel()
41. }
42.
```

```
43. func (producer *Producer) StartWork() {
44.     go func() {
45.         i := 0
46.         for {
47.             i++
48.             select {
49.             case <-producer._ctx.Done():
50.                 {
51.                     close(producer._exited)
52.                     return
53.                 }
54.             case msgChan <- i:
55.                 fmt.Println("producer produce number is ", i)
56.             }
57.
58.             time.Sleep(50 * time.Millisecond)
59.         }
60.     }()
61. }
```

我们通过init函数中只调用一次的方式初始化了producer，之后生成了一个名为_exited的channel，用来通知Join返回。
同样我们还初始化了一个可以取消的context，主要是在Exit函数内调用cancel取消上下文，会触发 `StartWork` 中 `producer._ctx.Done()` 进而终止生产工作，再发出退出信号，达到优雅退出的效果。

类似的消费者也是相似逻辑

```
1. package message
2.
3. import (
4.     "context"
5.     "fmt"
6.     "sync"
7.     "time"
8. )
9.
10. var consumer *Consumer = nil
11. var consumer_once sync.Once
12.
13. func init() {
14.     // Consumer1 = new(Consumer)
15.     //类似于C++ std::call_once
16.     consumer_once.Do(func() {
17.         consumer = new(Consumer)
18.         consumer._exited = make(chan struct{})
19.         consumer._ctx, consumer._cancel = context.WithCancel(context.Background())
20.         consumer.StartWork()
21.     })
22. }
23.
24. func ConsumerInst() *Consumer {
25.     return consumer
26. }
27.
28. type Consumer struct {
29.     _exited chan struct{}
30.     _ctx    context.Context
31.     _cancel context.CancelFunc
32. }
33.
34. func (consumer *Consumer) Join() {
35.     <-consumer._exited
36.     fmt.Println("consumer exited")
37. }
38.
39. func (consumer *Consumer) Exit() {
40.     consumer._cancel()
41. }
42.
```

```
43. func (consumer *Consumer) StartWork() {
44.     go func() {
45.         i := 0
46.         for {
47.             select {
48.             case <-consumer._ctx.Done():
49.                 {
50.                     close(consumer._exited)
51.                     return
52.                 }
53.             case i = <-msgChan:
54.                 fmt.Println("consumer consum number is ", i)
55.             }
56.
57.             time.Sleep(100 * time.Millisecond)
58.         }
59.     }()
60. }
```

C++ 风格的csp

C++是万能的，我们可以用C++实现一个类似于go的channel，采用csp模式解耦合，实现类似的生产者和消费者问题

```cpp
#include <iostream>
#include <queue>
#include <mutex>
#include <condition_variable>

template <typename T>
class Channel {
private:
    std::queue<T> queue_;
    std::mutex mtx_;
    std::condition_variable cv_producer_;
    std::condition_variable cv_consumer_;
    size_t capacity_;
    bool closed_ = false;

public:
    Channel(size_t capacity = 0) : capacity_(capacity) {}

    bool send(T value) {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_producer_.wait(lock, [this]() {
            // 对于无缓冲的channel，我们应该等待直到有消费者准备好
            return (capacity_ == 0 && queue_.empty()) || queue_.size() < capacity_ || closed_;
            });

        if (closed_) {
            return false;
        }

        queue_.push(value);
        cv_consumer_.notify_one();
        return true;
    }

    bool receive(T& value) {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_consumer_.wait(lock, [this]() { return !queue_.empty() || closed_; });

        if (closed_ && queue_.empty()) {
            return false;
        }

```

```
        value = queue_.front();
        queue_.pop();
        cv_producer_.notify_one();
        return true;
    }

    void close() {
        std::unique_lock<std::mutex> lock(mtx_);
        closed_ = true;
        cv_producer_.notify_all();
        cv_consumer_.notify_all();
    }
};

// 示例使用
int main() {
    Channel<int> ch(10);  // 10缓冲的channel

    std::thread producer([&]() {
        for (int i = 0; i < 5; ++i) {
            ch.send(i);
            std::cout << "Sent: " << i << std::endl;
        }
        ch.close();
        });

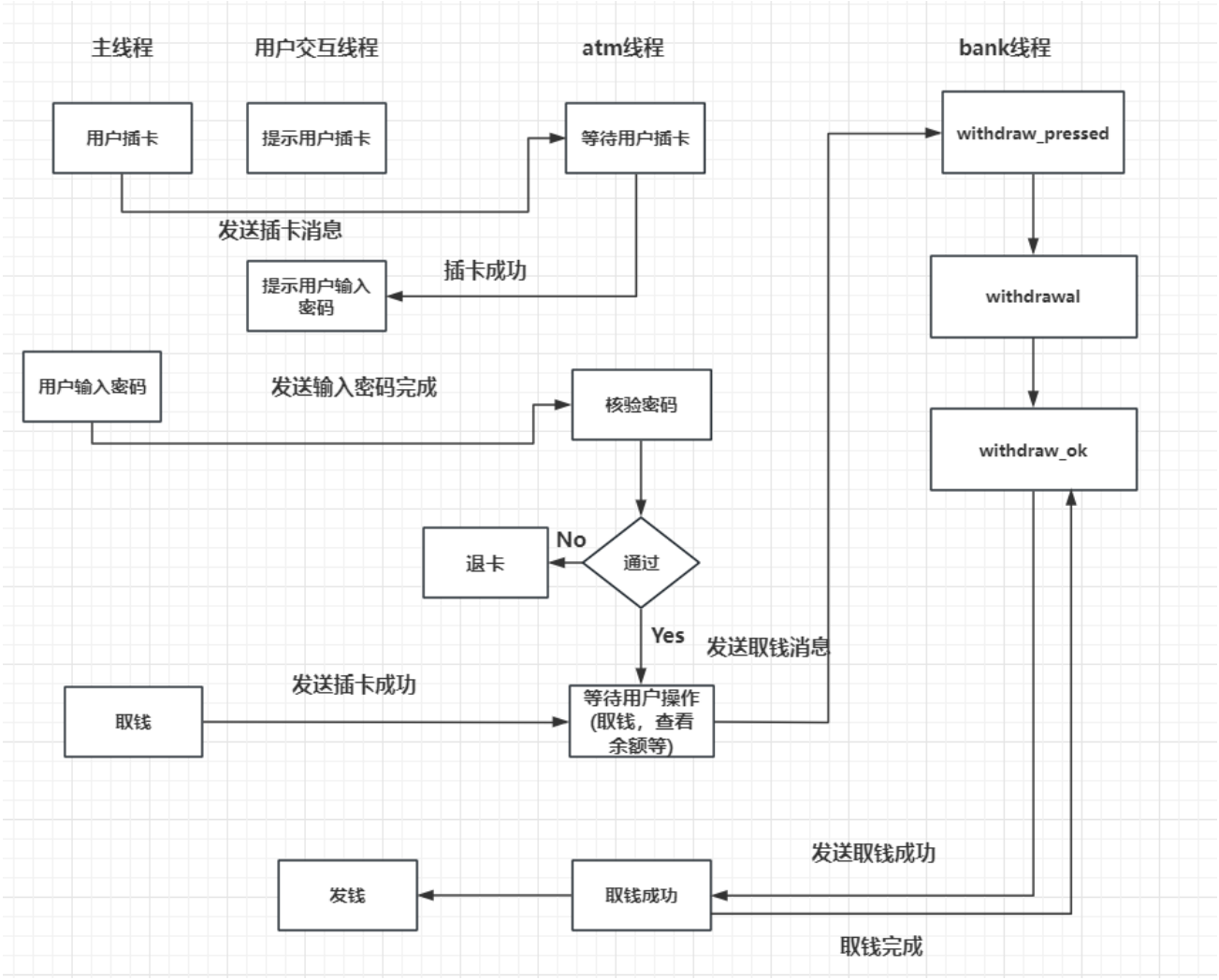
    std::thread consumer([&]() {
        std::this_thread::sleep_for(std::chrono::milliseconds(500)); // 故意延迟消费者开始消费
        int val;
        while (ch.receive(val)) {
            std::cout << "Received: " << val << std::endl;
        }
        });

    producer.join();
    consumer.join();
    return 0;
}
```

简单来说就是通过条件变量实现通信的阻塞和同步的。

利用csp思想实现取款逻辑

《C++并发编程实战》一书中提及了用csp思想实现atm机取款逻辑，我根据书中思想，整理了通信的示意图，书中部分代码存在问题，也一并修复了。



主函数实现

```
1. // Actor.cpp : 此文件包含 "main" 函数。程序执行将在此处开始并结束。
2. //
3.
4. #include <iostream>
5. #include "message.h"
6. #include "withdraw_msg.h"
7. #include "atm.h"
8. #include "dispatcher.h"
9. #include "bank_matchine.h"
10. #include "interface_matchine.h"
11.
12. int main()
13. {
14.     bank_machine bank;
15.     interface_machine interface_hardware;
16.     atm machine(bank.get_sender(), interface_hardware.get_sender());
17.     std::thread bank_thread(&bank_machine::run, &bank);
18.     std::thread if_thread(&interface_machine::run, &interface_hardware);
19.     std::thread atm_thread(&atm::run, &machine);
20.     messaging::sender atmqueue(machine.get_sender());
21.     bool quit_pressed = false;
22.     while (!quit_pressed)
23.     {
24.         char c = getchar();
25.         switch (c)
26.         {
27.         case '0':
28.         case '1':
29.         case '2':
30.         case '3':
31.         case '4':
32.         case '5':
33.         case '6':
34.         case '7':
35.         case '8':
36.         case '9':
37.             atmqueue.send(digit_pressed(c));
38.             break;
39.         case 'b':
40.             atmqueue.send(balance_pressed());
41.             break;
42.         case 'w':
```

```
43.             atmqueue.send(withdraw_pressed(50));
44.             break;
45.         case 'c':
46.             atmqueue.send(cancel_pressed());
47.             break;
48.         case 'q':
49.             quit_pressed = true;
50.             break;
51.         case 'i':
52.             atmqueue.send(card_inserted("acc1234"));
53.             break;
54.         }
55.     }
56.     bank.done();
57.     machine.done();
58.     interface_hardware.done();
59.     atm_thread.join();
60.     bank_thread.join();
61.     if_thread.join();
62. }
```

主函数中启动了三个线程，分别处理bank，machine以及interface的操作。
由于代码复杂解析来只列举atm类的实现

```
#pragma once
#include "dispatcher.h"
#include <functional>
#include <iostream>
class atm
{
    messaging::receiver incoming;
    messaging::sender bank;
    messaging::sender interface_hardware;
    void (atm::* state)();
    std::string account;
    unsigned withdrawal_amount;
    std::string pin;

    void process_withdrawal()
    {
        incoming.wait().handle<withdraw_ok, std::function<void(withdraw_ok const& msg)>,
            messaging::dispatcher >(
                [&](withdraw_ok const& msg)
                {
                    interface_hardware.send(
                        issue_money(withdrawal_amount));
                    bank.send(
                        withdrawal_processed(account, withdrawal_amount));
                    state = &atm::done_processing;
                }, "withdraw_ok").handle<withdraw_denied, std::function<void(withdraw_denied c
onst& msg)>>(
                    [&](withdraw_denied const& msg)
                    {
                        interface_hardware.send(display_insufficient_funds());
                        state = &atm::done_processing;
                    }, "withdraw_denied").handle<cancel_pressed, std::function<void(cancel_presse
d const& msg)>>(
                    [&](cancel_pressed const& msg)
                    {
                        bank.send(
                            cancel_withdrawal(account, withdrawal_amount));
                        interface_hardware.send(
                            display_withdrawal_cancelled());
                        state = &atm::done_processing;
                    }, "cancel_pressed"
                );
```

```
41.  }
42.  void process_balance()
43.  {
44.     incoming.wait()
45.        .handle<balance, std::function<void(balance const& msg)>,
46.        messaging::dispatcher>(
47.           [&](balance const& msg)
48.           {
49.              interface_hardware.send(display_balance(msg.amount));
50.              state = &atm::wait_for_action;
51.           },"balance"
52.           ).handle < cancel_pressed, std::function<void(cancel_pressed const& msg) >>(
53.              [&](cancel_pressed const& msg)
54.              {
55.                 state = &atm::done_processing;
56.              }, "cancel_pressed"
57.           );
58.  }
59.  void wait_for_action()
60.  {
61.     interface_hardware.send(display_withdrawal_options());
62.     incoming.wait()
63.        .handle<withdraw_pressed, std::function<void(withdraw_pressed const& msg)>,
64.        messaging::dispatcher>(
65.           [&](withdraw_pressed const& msg)
66.           {
67.              withdrawal_amount = msg.amount;
68.              bank.send(withdraw(account, msg.amount, incoming));
69.              state = &atm::process_withdrawal;
70.           }, "withdraw_pressed"
71.     ).handle < balance_pressed, std::function<void(balance_pressed const& msg) >>(
72.              [&](balance_pressed const& msg)
73.              {
74.                 bank.send(get_balance(account, incoming));
75.                 state = &atm::process_balance;
76.              }, "balance_pressed"
77.              ).handle<cancel_pressed, std::function<void(cancel_pressed const& msg) >>(
78.                 [&](cancel_pressed const& msg)
79.                 {
80.                    state = &atm::done_processing;
81.                 }, "cancel_pressed"
82.              );
```

```
}
void verifying_pin()
{
    incoming.wait()
        .handle<pin_verified, std::function<void(pin_verified const& msg)>,
        messaging::dispatcher>(
            [&](pin_verified const& msg)
            {
                state = &atm::wait_for_action;
            }, "pin_verified"
            ).handle<pin_incorrect, std::function<void(pin_incorrect const& msg)>>(
            [&](pin_incorrect const& msg)
            {
                interface_hardware.send(
                    display_pin_incorrect_message());
                state = &atm::done_processing;
            }, "pin_incorrect"
            ).handle<cancel_pressed, std::function<void(cancel_pressed const& msg)>>(
                    [&](cancel_pressed const& msg)
                    {
                        state = &atm::done_processing;
                    }, "cancel_pressed"
            );
}
void getting_pin()
{
    incoming.wait().handle<digit_pressed, std::function<void(digit_pressed const& msg)>,
        messaging::dispatcher>(
            [&](digit_pressed const& msg)
            {
                unsigned const pin_length = 6;
                pin += msg.digit;
                if (pin.length() == pin_length)
                {
                    bank.send(verify_pin(account, pin, incoming));
                    state = &atm::verifying_pin;
                }
            }, "digit_pressed"
            ).handle<clear_last_pressed, std::function<void(clear_last_pressed const& msg)>>(
            [&](clear_last_pressed const& msg)
            {
                if (!pin.empty())
```

```
125.                {
126.                    pin.pop_back();
127.                }
128.            }, "clear_last_pressed"
129.            ).handle<cancel_pressed, std::function<void(cancel_pressed const& msg)>>(
130.                [&](cancel_pressed const& msg)
131.                {
132.                    state = &atm::done_processing;
133.                }, "cancel_pressed"
134.            );
135.    }
136.
137.    void waiting_for_card()
138.    {
139.        interface_hardware.send(display_enter_card());
140.        incoming.wait().handle<card_inserted, std::function<void(card_inserted const& msg)>,
141.            messaging::dispatcher>(
142.                [&](card_inserted const& msg)
143.                {
144.                    account = msg.account;
145.                    pin = "";
146.                    interface_hardware.send(display_enter_pin());
147.                    state = &atm::getting_pin;
148.                }, "card_inserted"
149.        );
150.    }
151.    void done_processing()
152.    {
153.        interface_hardware.send(eject_card());
154.        state = &atm::waiting_for_card;
155.    }
156.    atm(atm const&) = delete;
157.    atm& operator=(atm const&) = delete;
158. public:
159.    atm(messaging::sender bank_,
160.        messaging::sender interface_hardware_) :
161.        bank(bank_), interface_hardware(interface_hardware_)
162.    {}
163.    void done()
164.    {
165.        get_sender().send(messaging::close_queue());
166.    }
```

```
167.    void run()
168.    {
169.        state = &atm::waiting_for_card;
170.        try
171.        {
172.            for (;;)
173.            {
174.                (this->*state)();
175.            }
176.        }
177.        catch (messaging::close_queue const&)
178.        {
179.        }
180.    }
181.    messaging::sender get_sender()
182.    {
183.        return incoming;
184.    }
185. };
```

atm 主要功能就是通过状态机不断地切换状态监听想要处理的函数。

详细源码可参考https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day08-actor/Actor (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day08-actor/Actor)

总结

本文讲述了Actor设计模式和CSP设计模式，并通过go和C++等语言给大家展示了csp并发设计的demo，最后通过讲解《C++并发编程实战》中取款的案例，展示了csp的用法。

详细源码

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day08-actor/Actor (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day08-actor/Actor)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2569) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(520) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)